STACK

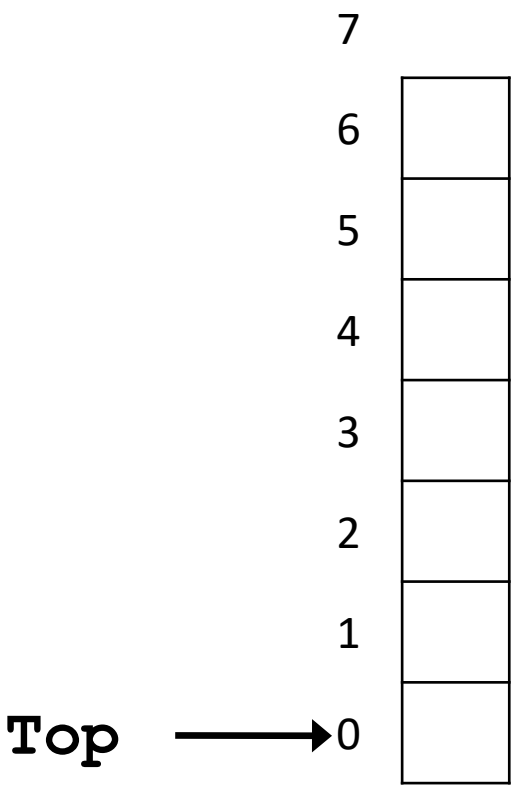
- A stack or LIFO (last in, first out) is an abstract data type that serves as a collection of elements, with two principal operations:
  - push adds an element to the collection;
  - pop removes the last (top of the stack) element that was added.
- Bounded capacity
  - If the stack is full and does not contain enough space to accept an entity to be pushed, the stack is then considered to be in an overflow state.
  - A pop either reveals previously concealed items or results in an empty stack – which means no items are present in stack to be removed.
- Non-Bounded capacity
  - Dynamically allocate memory for stack. No overflow.

STACK

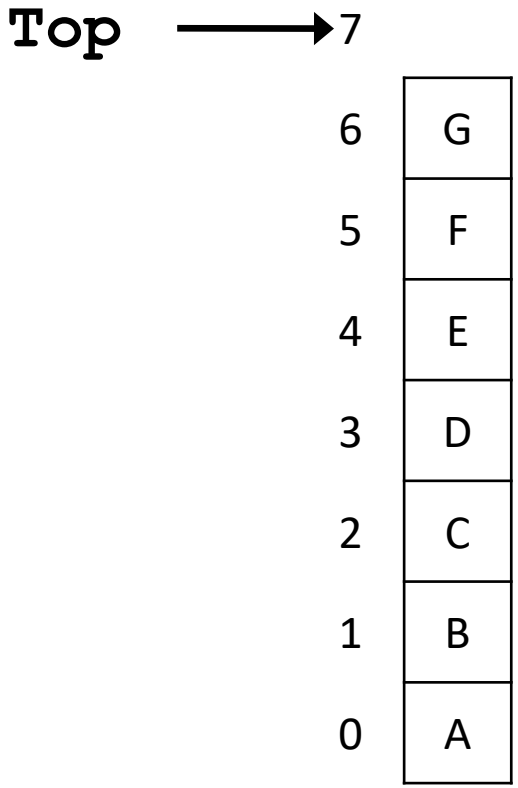
```
int Stack[100], Top=0, MaxSize=100; // Stack holds the elements;
// Top is the index of Stack always pointing to the first/top element of the stack.
bool IsEmpty( void ); // returns True if stack has no element
bool IsFull( void ); // returns True if stack full
bool Push( const int Element ); // inserts Element at the top of the stack
bool Pop( int &Element ); // deletes top element from stack into Element
bool TopElement( int &Element ); // gives the top element in Element
void Show( void ); // prints the whole stack
```

```
bool IsEmpty( void ){
      // returns True if stack has no element
      return (Top == 0);
}
```

Considering **MaxSize** = 7

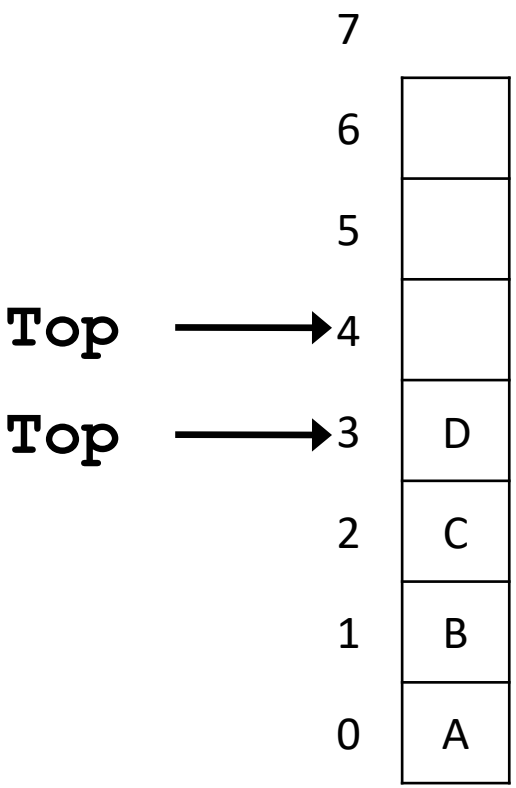


```
bool IsFull( void ){
      // returns True if stack full
      return ( Top == MaxSize );
}
```



Considering **MaxSize** = 7

```
bool Push( const int Element ){
      // inserts Element at the top of the stack
      if( IsFull( ) ) { cout << "Stack is Full\n"; return false; }
      // push element if there is space
      Stack[ Top++ ] = Element;
      return true;
}
```

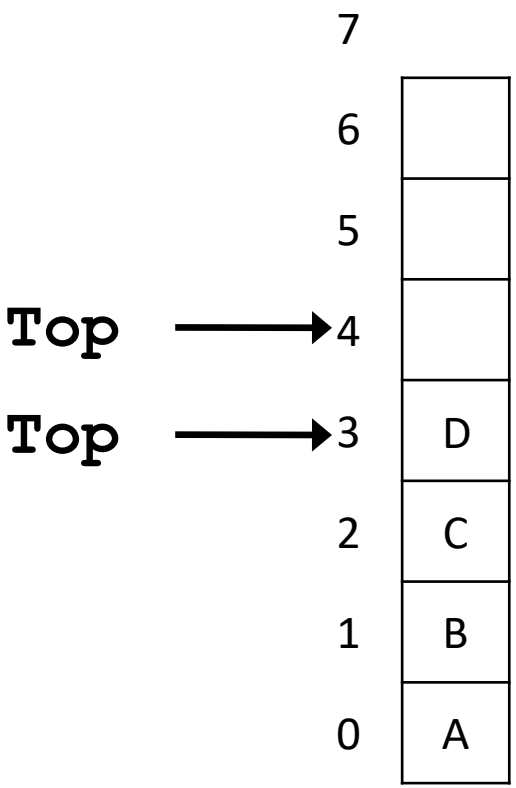


Considering **MaxSize** = 7

There are 3 elements inside Stack

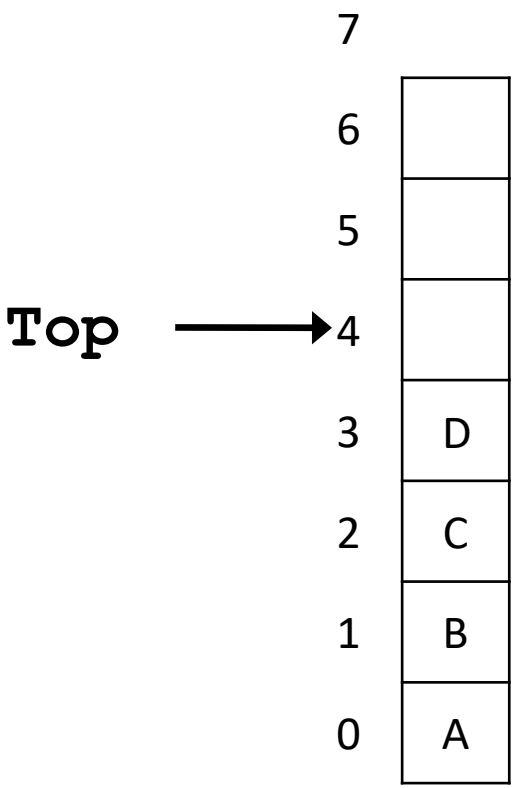
So next element will be pushed at index 3

```
bool Pop( int &Element ){
      // deletes top element from stack and puts it in Element
      if( IsEmpty() ) { cout << "Stack empty\n"; return false; }
      Element = Stack[ --Top ];
      return true;
}
```



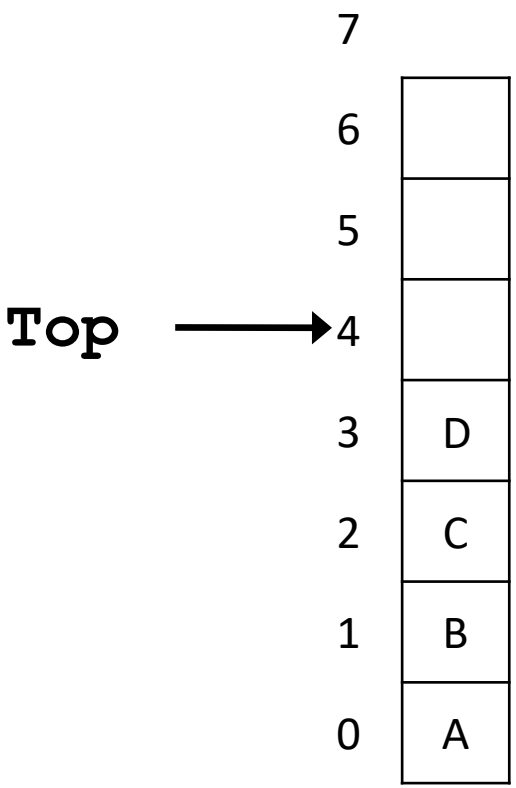
Considering **MaxSize** = 7
There are 4 elements inside Stack
So element will be popped from index 3

```
bool TopElement( int &Element ){
      // gives the top element in Element
      if( IsEmpty() ) { cout << "Stack empty\n"; return false; }
      Element = Stack[ Top - 1 ];
      return true;
}
```



Considering **MaxSize** = 7
There are 4 elements inside Stack
So top element will be at index 3

```
void Show( void ){
      // prints the whole stack
      if( IsEmpty() ) { cout << "Stack empty\n"; return; }
      for( int i=Top-1; i>=0; i-- ) cout << Stack[i] << endl;
}
```



Considering **MaxSize** = 7
There are 4 elements inside Stack
So element will be shown from index 3 down to index 0.

CREATING A `class` FOR STACK

```
class MyStack{
   int Stack[100], Top, MaxSize;
public:
   //Initializing stack
   MyStack( int Size = 100 ){     MaxSize = Size; Top = 0;}
   bool IsEmpty( void );
   bool IsFull( void );
   bool Push( const int Element );
   bool Pop( int &Element );
   bool TopElement( int &Element );
   void Show( void );
   void Reset( void ){  Top = 0;  } //Re-start the stack
};
```

Stack Using Dynamic Memory Allocation

```
class MyStack{
   int *Stack, Top, MaxSize;
public:
   MyStack( int );
   ~MyStack( void );
   bool IsEmpty( void );
   bool IsFull( void );
   bool Push( const int );
   bool Pop( int & );
   bool TopElement( int & );
   void Show( void );
   void Reset( void ){ Top = 0; }
   void Resize( int ); //Resize the stack
};
```

❁The Constructor will create the array dynamically, Destructor will release it.

```
MyStack::MyStack( int Size = 100 ){
   MaxSize = Size; // get Size
   Stack = new int[ MaxSize ]; // create array acordingly
   Top = 0; // start the stack
}
MyStack::~MyStack( void ){
   delete [] Stack; // release the memory for stack
}
```

- `Resize` creates a new array dynamically, copies all the element from the previous stack, releases the old array, and makes the pointer `Stack` point to the new array.
- By default increase 100, user can define the additional size. Use negative size to decrease the array.

```
void Resize( int Size = 100 ){
      // creates a new stack with MaxSize + Size
      int *S = new int[ MaxSize + Size ];
      // copy the elements from old to new stack
      for( int i=0; i<MaxSize; i++ ) S[i] = Stack[i];
      MaxSize += Size; // MaxSize increases by Size
      delete [] Stack; // release the old stack
      Stack = S; // assign Stack with new stack.
}
void Push( const int Element ){
      // inserts Element at the top of the stack
      if( StackFull( ) )    Resize( ); // increase size if full
      Stack[ ++Top ] = Element;
}
```

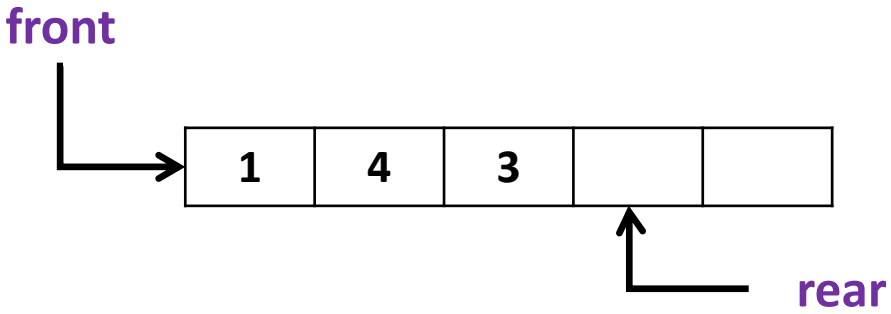
GENERIC STACK

```
template <typename T>
class MyStack{
   T *Stack;
   int Top, MaxSize;
public:
   MyStack( int );
   ~MyStack( void );
   bool IsEmpty( void );
   bool IsFull( void );
   bool Push( const T );
   bool Pop( T & );
   bool TopElement( T & );
   void Show( void );
   void Reset( void ){    Top = 0; }
   void Resize( int ); //Resize the stack
};
```

QUEUE

- A queue is a waiting line – seen in daily life
- A line of people waiting for a bank teller
- A line of cars at a toll both
- Queue data structure is like a container with both end opening.
  - An end called **rear**
  - Another end is **front**
- This mechanism is called First-In-First-Out (FIFO).
- Some of the applications are :
  - Device queue, printer queue, keystroke queue, etc.

QUEUE IN COMPUTER LANGUAGE

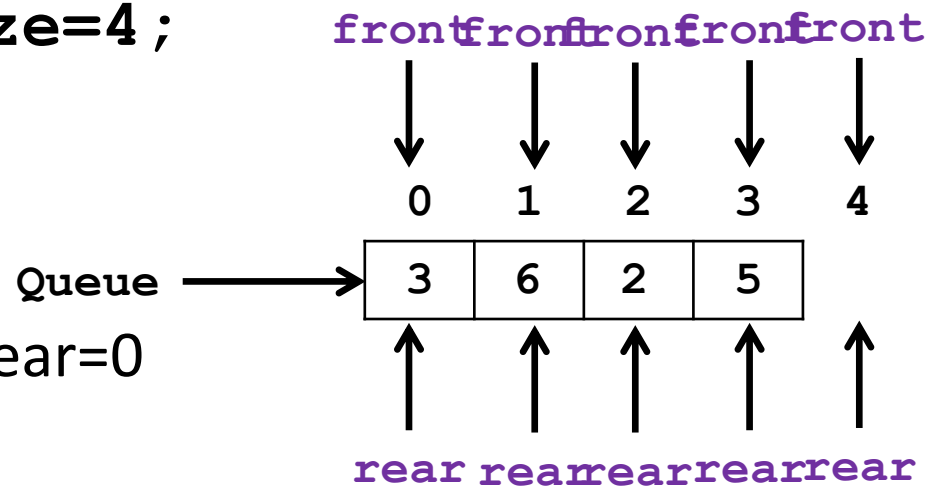


- A queue is a sequence of data elements
- In the sequence
  - Items can be added only at the **rear**
  - Items can be removed only at the other end, **front**
- Basic operations
  - Check **`IsEmpty`**
  - Check **`IsFull`**
  - **`EnQueue`** (add element to back i.e. at the rear)
  - **`DeQueue`** (remove element from the front)
  - **`FrontValue`** (retrieve value of element from front)
  - **`ShowQueue`** (print all the values of stack from front to rear)

QUEUE - OPERATION

- Check **`IsEmpty`**
- Check **`IsFull`**
- **`EnQueue`** (add element to back i.e. at the rear)
- **`DeQueue`** (remove element from the front)
- **`FrontValue`** (retrieve value of element from front)
- **`ShowQueue`** (print all the values of stack from front to rear)

- **`Queue[4],MaxSize=4;`**

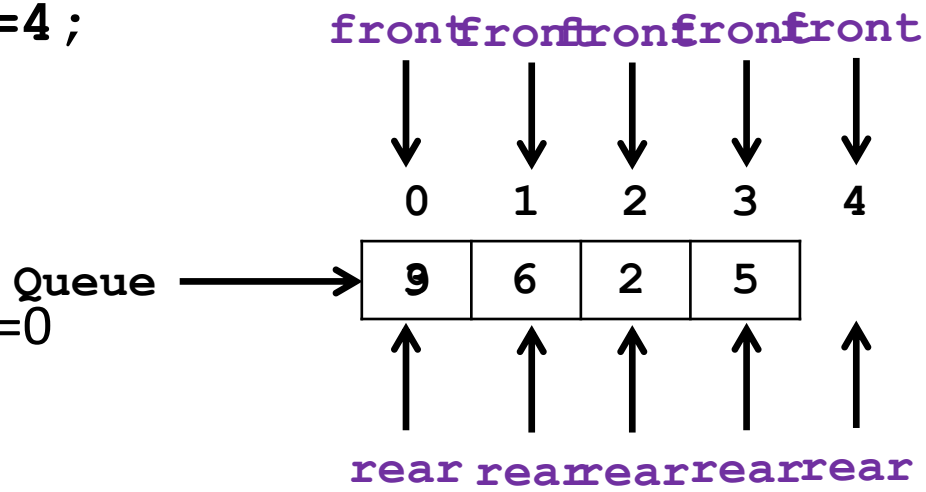


- Initialize( ) → front=rear=0
- EnQueue(3)
- EnQueue(6)
- EnQueue(2)
- EnQueue(5)
- EnQueue(9) → Queue Full (rear==MaxSize)
- DeQueue(&v) → 3
- DeQueue(&v) → 6
- DeQueue(&v) → 2
- DeQueue(&v) → 5
- DeQueue(&v) → Queue Empty (front==rear)

Circular Queue - Operation

- Check **`IsEmpty`**
- Check **`IsFull`**
- **`EnQueue`** (add element to back i.e. at the rear)
- **`DeQueue`** (remove element from the front)
- **`FrontValue`** (retrieve value of element from front)
- **`ShowQueue`** (print all the values of stack from front to rear)

- **`Queue[4],MaxSize=4;`**



- Initialize( ) → front=rear=0
- EnQueue(3)
- EnQueue(6)
- EnQueue(2)
- DeQueue(&v) → 3
- DeQueue(&v) → 6
- EnQueue(5) → (rear==MaxSize)→rear=0
- EnQueue(9)
- DeQueue(&v) → 2
- DeQueue(&v) → 5 (front==MaxSize) → front=0
- DeQueue(&v) → 9
- DeQueue(&v) → Queue Empty (front==rear)